

Code Review Summary Report

Overview

This assignment focused on performing a formal code review of a Boggle solver implementation. Code review is the systematic examination of source code by peers to identify mistakes, improve structure, and enhance overall quality before software is finalized. As discussed in class, finding bugs early is much more cost-effective than fixing them later in production.

For this assignment, I implemented a Boggle solver using a class-based design. My solver uses a Depth-First Search (DFS) algorithm with backtracking to explore all possible word paths in the grid. To improve efficiency, I created a prefix set from the dictionary so the solver can stop exploring a path early if it cannot form a valid word. This pruning technique prevents unnecessary recursive calls and improves performance.

Feedback

My partner gave helpful and constructive feedback throughout the file. One key suggestion was to mention explicitly in the docstring that the solver uses DFS. While the code structure implies it, adding this detail would make the documentation clearer and more complete for someone reading it for the first time. They also suggested using type hints (like `Optional[list]`) in the constructor to improve readability and make the code more self-documenting. This would help clarify expected data types.

These suggestions would make the code more concise while maintaining functionality. They also pointed out that it might be worth handling empty strings in the dictionary and optionally documenting that resetting the grid or dictionary after initialization does not automatically trigger solving again. Overall, my partner complimented the clarity of variable names, defensive initialization, predictable sorted output, and readable validation logic. The feedback was encouraging but also included concrete improvement suggestions.

When reviewing my partner's code, I focused on structure, clarity, and correctness. I noted that their class structure was clean and well-organized, which made the code easy to follow. However, I suggested defining the `getSolution()` method more clearly to avoid potential crashes if it were called before solving. I also recommended handling edge cases such as empty grids or dictionaries to prevent unexpected behavior. Overall, I emphasized that the code had a solid foundation and would come together well after implementing the full solving logic.

Improvements

After receiving feedback, I made small but meaningful improvements. First, I updated documentation wording to be clearer and more specific. I made sure the docstring accurately described what the solver does and how it works. Second, I corrected minor grammar issues to improve professionalism and readability. I also reviewed areas where list comprehensions could improve conciseness. While I kept some loops for readability (since clarity can be more important than brevity), I now understand how comprehensions could simplify sections of the code. These changes improved clarity and polish without changing the solver's core functionality.

Static Analysis Results & Regression Testing

During review, I corrected small grammar errors and improved formatting consistency. There were no major linting issues or structural warnings. Most changes involved improving readability and ensuring consistent style across methods.

After making changes, I reran the solver using the sample grid and dictionary in the `main()` function. I verified that:

- Valid words were still found correctly
- Words were returned in sorted order
- Special tiles like "IE" were handled correctly
- No new errors were introduced

Since the core logic was unchanged, I mainly verified that formatting and documentation changes did not affect execution. The output remained consistent before and after the revisions.

Reflection

This assignment helped me understand how important peer review is in professional software development. Even when code works, there are always improvements that can be made in readability, efficiency, and documentation.

One thing I learned is that small stylistic suggestions (like using list comprehensions or adding type hints) can make code cleaner and more maintainable. I also learned that explaining your algorithm clearly in documentation is just as important as writing correct code.

Code reviews are not just about finding errors. They are about improving clarity, performance, and long-term maintainability. Seeing how my partner approached feedback helped me understand how collaborative software development works in real-world environments.